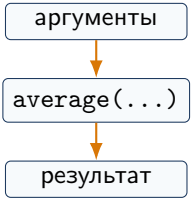
Функции

Введение в ПИ (C++)

Functions

Именованный рецепт

- Повторное использование
- Читаемое имя намерения
- Удобнее проверять по частям
- Короткий `main`



Объявление и определение

```cpp
// declaration (prototype)
double average(double a, double b);

int main() {
    std::cout << average(3.0, 5.0) << "\n";
}

// definition
double average(double a, double b) {
    return (a + b) / 2.0;
}
```

Параметры и return

```
int clamp(int x, int lo, int hi) {
    if (x < lo) return lo;
    if (x > hi) return hi;
    return x;
}

void greet(const std::string& name) {
    std::cout << "Hi, " << name << "\n";
}
```

`void` — нет возвращаемого значения.

Передача по значению

По умолчанию параметр — **копия** аргумента.

Изменения копии внутри функции не видны снаружи.

Ссылки — на лекции 06; пока: большие объекты копировать дорого.

Область видимости *(scope)*

- Локальная переменная живёт в блоке `\{...\}`
- Параметры — локальны для функции
- Имя во внутреннем блоке может скрыть внешнее *(shadowing)* — избегайте

Перегрузка *(overloading)*

```
1 int max_of(int a, int b);
2 double max_of(double a, double b);
```

Одинаковое имя, разные списки параметров.

Только тип возврата — **недостаточно** для перегрузки.

Викторина 1 — что напечатает?

Викторина

Результат?

```
1 void inc(int x) { x = x + 1; }
2 int main() {
3     int a = 3;
4     inc(a);
5     std::cout << a;
6 }
```

1. 3
2. 4
3. ошибка компиляции
4. неопределённое поведение

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 1

Ответ

1. Передача по значению: меняется копия, а остаётся 3.

Викторина 2 — перегрузка

Викторина

Можно ли так перегрузить?

```
1 int f(int x);
2 double f(int x);  // different return type only
```

1. Да, компилятор различит по контексту использования
2. Нет, списки параметров одинаковы
3. Да, если функции в разных файлах
4. Да, если одна `static`

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 2

Ответ

2. Перегрузка требует различных параметров, не только return type.